

Vue 一复习要点

1 Vue 简介

1.1 Vue 的由来

[p.4]

- > 由尤雨溪 (Evan You) 于 2014 年创建 [p.4]
- > 在使用 AngularJS 后，提取其核心功能 (ViewModel)，创建了更轻量、灵活的框架 [p.4]
- > 创建理念：”我想要一个能够轻松上手，同时又足够强大的框架” —Evan You [p.4]
- > 定位：渐进式 JavaScript 框架 (Progressive JavaScript Framework) [p.4]
- > 官网：<https://cn.vuejs.org> [p.4]

1.2 Vue 核心特性

[p.5]

特性	描述	解决的问题
渐进式	从简单到复杂逐步引入	降低学习门槛，适合各种规模
响应式	数据变化自动更新视图	避免手动 DOM 操作
组件化	UI 拆分为可复用组件	提高代码复用性
模板语法	HTML-based 模板，易学易用	降低学习成本
双向绑定	v-model 实现双向同步	简化表单处理

1.3 Vue vs React 对比

[p.6]

维度	Vue	React
模板语法	HTML-based 模板	JSX
响应式	基于 Proxy 的响应式系统	基于 setState 的状态更新
数据绑定	双向绑定 (v-model)	单向数据流
学习曲线	平缓，渐进式	中等，需学 JSX
生态系统	Vue Router、Pinia	React Router、Redux
灵活性	中等	高，更接近原生 JS

2 Vue 模板语法

2.1 指令系统

[p.8–12]

指令	作用	示例
v-text	纯文本绑定	
v-html	HTML 内容绑定	<div v-html="htmlContent"></div>
v-bind	属性绑定	
v-on	事件绑定	<button v-on:click="handleClick">
v-model	双向绑定	<input v-model="username" />
v-if	条件渲染（销毁/重建）	<div v-if="show"> 显示 </div>
v-for	列表渲染	<li v-for="item in list">{{ item }}
v-show	条件显示（display 切换）	<div v-show="visible"> 可见 </div>

2.2 v-if vs v-show

[p.11]

场景	v-if	v-show
条件频繁切换	不推荐	推荐
条件很少改变	推荐	不推荐
需要销毁子组件	推荐（支持）	不适用
实现方式	DOM 销毁/重建	CSS display 切换
性能影响	切换成本高	初始渲染成本（始终渲染）

3 Vue 组件与数据传递

3.1 Vue 单文件组件

[p.14]

- > 组件是 Vue 应用的基本构建块，封装 HTML + CSS + JavaScript [p.14]
- > .vue 单文件组件三大组成 [p.14]:
- <template>: HTML 结构，只能有一个根元素

- <script>: JavaScript 逻辑，setup 是组合式 API (Composition API)

- <style>: 样式，scoped 表示只作用于当前组件

3.2 组件间数据传递

[p.15–16]

- > 父 → 子：通过 **Props** 传递 [p.15]
- 可指定类型 (String/Number/Boolean/Array/Object/Function)、是否必填 (required)、默认值 (default)、自定义验证 (validator)
- > 子 → 父：通过 **emit** 事件传递 [p.16]

4 Vue 响应式系统

4.1 响应式原理

[p.18]

- > Vue 3 使用 **Proxy** 对象拦截数据的读取和写入 [p.18]

> 三步机制 [p.18]:

1. 读取数据时建立**依赖关系**（依赖收集 / Dependency Tracking）
2. 修改数据时触发**更新**（派发更新 / Dispatch Update）
3. 视图自动重新渲染

> 数据两种表达方式 [p.18]:

- `ref()`: 基本类型数据（数字、字符串等）
- `reactive()`: 对象类型数据

4.2 计算属性 `computed`

[p.20–23]

> 基于依赖自动计算的值，具有**缓存特性**（依赖不变时不重新计算） [p.20]

> `computed` vs `methods` [p.23]:

对比维度	<code>computed</code>	<code>methods</code>
缓存	依赖变化时重新计算	每次调用都执行
调用方式	<code>{{ fullName }}</code> （无括号）	<code>{{ getFullName() }}</code> （有括号）
返回值	必须返回值	可返回可不返回
副作用	不应该有副作用	可以有副作用
使用场景	派生值计算、过滤、格式化	事件处理、数据获取

4.3 生命周期钩子

[p.24]

钩子函数	触发时机	常用场景
<code>onBeforeMount</code>	组件挂载前	准备工作
<code>onMounted</code>	组件挂载后	数据初始化、DOM 操作
<code>onBeforeUpdate</code>	组件更新前	保存状态
<code>onUpdated</code>	组件更新后	更新 DOM
<code>onBeforeUnmount</code>	组件卸载前	确认操作
<code>onUnmounted</code>	组件卸载后	清理资源、取消订阅

5 前端其他知识

5.1 TypeScript

[p.27–31]

> JS 痛点 [p.27]: 动态类型语言，灵活性带来维护困难

> **TypeScript** [p.28]: Microsoft 2012 年发布，JS 的**超集** (Superset)，添加可选静态类型系统。可与 React/Vue 配合使用 [p.30–31]

5.2 服务端渲染 SSR

[p.33–36]

> SPA 首屏加载慢 + SEO 不友好的解决方案 [p.33]

> 在服务端完成首次渲染，客户端接管后续交互（同构应用） [p.34–36]

5.3 性能优化

[p.38–42]

> 代码层面 [p.38–40]: 减少不必要的渲染、合理使用 `computed/watch`、懒加载

> 资源优化 [p.41]: 代码分割 (Code Splitting)、Tree Shaking、压缩

> 网络层面 [p.42]: CDN、HTTP/2、缓存策略

6 本章小结与考点

[p.43]

- > **Vue 简介** [p.4-6]: 渐进式框架、Vue vs React 对比
- > **模板语法** [p.8-12]: 8 种核心指令、v-if vs v-show
- > **组件化** [p.14-16]: .vue 单文件组件、Props 向下/Emit 向上
- > **响应式系统** [p.18-24]: Proxy 原理、ref/reactive、computed vs methods、生命周期钩子
- > **前端其他** [p.27-42]: TypeScript、SSR、性能优化

高频考点: (1) Vue 渐进式框架的定位和五大核心特性 [p.4-5]; (2) Vue vs React 六大维度对比 [p.6]; (3) Vue 八大模板指令及其作用 [p.9]; (4) v-if vs v-show 的区别和选择依据 [p.11]; (5) .vue 单文件组件三大组成 [p.14]; (6) Vue 3 基于 Proxy 的响应式原理 [p.18]; (7) computed vs methods 五大对比维度 [p.23]; (8) Vue 生命周期六大钩子函数 [p.24]